

```
1: /**
2:  * @file claves_recursivo.c
3:  * @brief Generador recursivo de claves basado en diccionario y reglas
4:  * @author Asistente IA
5:  * @date 2026
6:  *
7:  * Este programa genera todas las combinaciones posibles de claves seg n:
8:  * - Un diccionario de palabras (caracteres individuales)
9:  * - Reglas que contienen '#' (palabra) y '0' (d gito)
10:  *
11:  * La generaci n es TOTALMENTE RECURSIVA, sin usar break/continue en ciclos.
12:  */
13:
14: #include <stdio.h>
15: #include <string.h>
16:
17: /* === CONSTANTES === */
18:
19: /** @brief N mero m ximo de claves que se pueden generar */
20: #define MAX_CLAVES 10000
21:
22: /** @brief Longitud m xima de cada clave generada */
23: #define MAX_CLAVE_LEN 100
24:
25: /* === VARIABLES GLOBALES === */
26:
27: /** @brief Arreglo para almacenar todas las claves generadas */
28: char resultados[MAX_CLAVES][MAX_CLAVE_LEN];
29:
30: /** @brief Contador del n mero total de claves generadas */
31: int total = 0;
32:
33: /* === FUNCIONES === */
34:
35: /**
36:  * @brief Genera recursivamente todas las combinaciones de claves
37:  *
38:  * Esta funci n recorre la regla car cter por car cter. Cuando encuentra:
39:  * - '#' : Itera sobre todas las palabras del diccionario
40:  * - '0' : Itera sobre todos los d gitos (0-9)
41:  * - Otro car cter: Lo copia literalmente
42:  *
43:  * La funci n es puramente recursiva: cada llamada procesa un car cter
44:  * de la regla y se llama a s  misma para el siguiente.
45:  *
46:  * @param regla Cadena con el patr n de la regla (ej: "#0", "0#0", "pre#0fijo
47:  *
48:  * @param posRegla  ndice actual en la regla (comienza en 0)
49:  * @param actual B fer donde se construye la clave actual
50:  * @param posActual  ndice actual en el b fer de la clave
51:  * @param palabras Cadena con todas las palabras del diccionario (ej: "r2")
52:  * @param numPalabras N mero de palabras en el diccionario
53:  *
54:  * @pre El arreglo 'resultados' debe tener suficiente espacio
55:  * @pre El b fer 'actual' debe tener al menos MAX_CLAVE_LEN de tama o
56:  * @post Se a aden las claves generadas al arreglo 'resultados'
57:  * @post Se incrementa 'total' con la cantidad de claves generadas
58:  *
59:  * @note NO se usa break ni continue en ning n ciclo
60:  * @note La recursi n termina cuando se alcanza el fin de la regla
61:  */
62: void generar(const char *regla, int posRegla, char *actual, int posActual,
63:             const char *palabras, int numPalabras) {
64:     /* =====
65:     * CASO BASE 1: Llegamos al final de la regla
66:     * =====
67:     * Cuando posRegla apunta al car cter nulo ('\0'), significa que
68:     * ya procesamos todos los caracteres de la regla.
69:     * En este punto, 'actual' contiene una clave completa.
```

```
70:  */
71:  if (regla[posRegla] == '\0') {
72:      /* Terminar la cadena con el carácter nulo */
73:      actual[posActual] = '\0';
74:
75:      /* Guardar la clave generada en el arreglo de resultados */
76:      strcpy(resultados[total], actual);
77:
78:      /* Incrementar el contador de claves */
79:      total++;
80:
81:      /* Retornar para continuar generando otras combinaciones */
82:      return;
83:  }
84:
85:  /* =====
86:  * VERIFICACIÓN DE SEGURIDAD: Evitar desbordamiento del arreglo
87:  * =====
88:  * Si ya generamos el máximo de claves permitidas, detenemos
89:  * la recursión para evitar segmentation fault.
90:  */
91:  if (total >= MAX_CLAVES) {
92:      return;
93:  }
94:
95:  /* =====
96:  * CASO RECURSIVO 1: El carácter actual es '#'
97:  * =====
98:  * '#' indica que en esta posición debe ir UNA palabra del diccionario.
99:  * Para CADA palabra disponible, la agregamos y continuamos recursivamente.
100:  *
101:  * Ejemplo: Si palabras = "r2", entonces probamos:
102:  *   - Primero 'r', luego seguimos con el resto de la regla
103:  *   - Luego '2', luego seguimos con el resto de la regla
104:  */
105:  if (regla[posRegla] == '#') {
106:      /* Recorrer todas las palabras del diccionario */
107:      /* NOTA: No se usa break, el ciclo se completa siempre */
108:      for (int i = 0; i < numPalabras; i++) {
109:          /* Colocar la palabra (carácter) en la clave actual */
110:          actual[posActual] = palabras[i];
111:
112:          /*
113:           * LLAMADA RECURSIVA:
114:           * - Avanzamos al siguiente carácter de la regla (posRegla + 1)
115:           * - Avanzamos al siguiente espacio en la clave (posActual + 1)
116:           * - El resto de parámetros se mantienen igual
117:           */
118:          generar(regla, posRegla + 1, actual, posActual + 1,
119:                 palabras, numPalabras);
120:      }
121:  }
122:  /* =====
123:  * CASO RECURSIVO 2: El carácter actual es '0'
124:  * =====
125:  * '0' indica que en esta posición debe ir un DÍGITO (0-9).
126:  * Para CADA dígito posible, lo agregamos y continuamos recursivamente.
127:  *
128:  * El orden ascendente (0,1,2,...,9) garantiza el orden requerido
129:  * en el problema.
130:  */
131:  else if (regla[posRegla] == '0') {
132:      /* Recorrer todos los dígitos del 0 al 9 */
133:      /* NOTA: El ciclo siempre completa todas las iteraciones (sin break) */
134:      for (int digito = 0; digito <= 9; digito++) {
135:          /* Convertir el número entero a carácter ('0' = 48 en ASCII) */
136:          actual[posActual] = '0' + digito;
137:
138:          /* LLAMADA RECURSIVA para el resto de la regla */
139:          generar(regla, posRegla + 1, actual, posActual + 1,
```

```
140:         palabras, numPalabras);
141:     }
142: }
143: /* =====
144:  * CASO RECURSIVO 3: Car cter literal (fijo)
145:  * =====
146:  * Si el car cter no es '#' ni '0', entonces es un car cter literal
147:  * que debe aparecer tal cual en todas las claves generadas.
148:  *
149:  * Ejemplo: En la regla "pre#0", 'p', 'r', 'e' son caracteres literales.
150:  */
151: else {
152:     /* Copiar el car cter literal tal cual aparece en la regla */
153:     actual[posActual] = regla[posRegla];
154:
155:     /* LLAMADA RECURSIVA para el resto de la regla */
156:     generar(regla, posRegla + 1, actual, posActual + 1,
157:             palabras, numPalabras);
158: }
159:
160: /* =====
161:  * NOTA SOBRE LA RECURSI N:
162:  * =====
163:  * No hay un "caso else" adicional porque todas las posibilidades
164:  * est n cubiertas: '#', '0', o cualquier otro car cter.
165:  *
166:  * La recursi n es profunda: la profundidad m xima es la longitud
167:  * de la regla (hasta 256 caracteres seg n el problema).
168:  *
169:  * El n mero total de llamadas recursivas es:
170:  * Suma de combinaciones = (numPalabras^num_hashes) * (10^num_ceros)
171:  */
172: }
173:
174: /**
175:  * @brief Funci n wrapper que inicia la generaci n de claves
176:  *
177:  * @param regla      Regla a procesar (ej: "#0")
178:  * @param palabras  Diccionario de palabras como cadena (ej: "r2")
179:  * @param numPalabras N mero de palabras en el diccionario
180:  *
181:  * @return int N mero de claves generadas
182:  *
183:  * @post El arreglo global 'resultados' contiene las claves generadas
184:  * @post La variable global 'total' se actualiza con el n mero de claves
185:  */
186: int generarClaves(const char *regla, const char *palabras, int numPalabras) {
187:     /* B fer para construir la clave actual */
188:     char clave[MAX_CLAVE_LEN];
189:
190:     /* Inicializar el contador global (por si se llama m ltiples veces) */
191:     total = 0;
192:
193:     /* Iniciar la generaci n recursiva */
194:     generar(regla, 0, clave, 0, palabras, numPalabras);
195:
196:     /* Retornar el n mero de claves generadas */
197:     return total;
198: }
199:
200: /**
201:  * @brief Imprime todas las claves generadas en el formato requerido
202:  *
203:  * El formato de salida es:
204:  * --          (dos guiones en una l nea)
205:  * clave1
206:  * clave2
207:  * ...
208:  * claveN
209:  */
```

```
210:  * @param mostrarTotal Si es diferente de 0, muestra además el total
211:  */
212: void imprimirClaves(int mostrarTotal) {
213:     printf("--\n");
214:
215:     for (int i = 0; i < total; i++) {
216:         printf("%s\n", resultados[i]);
217:     }
218:
219:     if (mostrarTotal) {
220:         printf("\nTotal: %d claves\n", total);
221:     }
222: }
223:
224: /**
225:  * @brief Función principal del programa
226:  *
227:  * Demuestra el uso de la función recursiva con diferentes ejemplos.
228:  *
229:  * @return int 0 si el programa termina correctamente
230:  */
231: int main() {
232:     printf("=====\n");
233:     printf("    GENERADOR RECURSIVO DE CLAVES\n");
234:     printf("=====\n\n");
235:
236:     /* =====
237:      * EJEMPLO 1: Caso del enunciado
238:      * =====
239:      * Diccionario: ['r', '2'] (2 palabras)
240:      * Regla: "#0" (palabra + dÃ-gito)
241:      *
242:      * Total esperado: 2 Ã\227 10 = 20 claves
243:      */
244:     printf("--- EJEMPLO 1: Regla '#0' con diccionario ['r','2'] ---\n");
245:
246:     char palabras1[] = {'r', '2'}; /* Arreglo de palabras */
247:     int numPalabras1 = 2;          /* Cantidad de palabras */
248:
249:     /* Generar las claves recursivamente */
250:     int total1 = generarClaves("#0", palabras1, numPalabras1);
251:
252:     /* Mostrar resultados */
253:     imprimirClaves(1); /* 1 = mostrar total */
254:
255:     /* =====
256:      * EJEMPLO 2: Regla "0#0" con diccionario ['A','B','C']
257:      * =====
258:      * Diccionario: ['A', 'B', 'C'] (3 palabras)
259:      * Regla: "0#0" (dÃ-gito + palabra + dÃ-gito)
260:      *
261:      * Total esperado: 10 Ã\227 3 Ã\227 10 = 300 claves
262:      */
263:     printf("\n--- EJEMPLO 2: Regla '0#0' con diccionario ['A','B','C'] ---\n");
264:
265:     char palabras2[] = {'A', 'B', 'C'};
266:     int numPalabras2 = 3;
267:
268:     int total2 = generarClaves("0#0", palabras2, numPalabras2);
269:
270:     printf("Total generado: %d claves (esperado: 300)\n", total2);
271:
272:     /* Mostrar solo las primeras 20 claves por brevedad */
273:     printf("Primeras 20 claves:\n");
274:     printf("--\n");
275:     for (int i = 0; i < 20 && i < total2; i++) {
276:         printf("%s\n", resultados[i]);
277:     }
278:     printf("...\n");
279: }
```

```
280: /* =====
281:  * EJEMPLO 3: Regla con mltiples '#'
282:  * =====
283:  * Diccionario: ['X', 'Y'] (2 palabras)
284:  * Regla: "##0" (palabra + palabra + dÃ-gito)
285:  *
286:  * Total esperado: 2 Ã\227 2 Ã\227 10 = 40 claves
287:  */
288: printf("\n--- EJEMPLO 3: Regla '##0' con diccionario ['X','Y'] ---\n");
289:
290: char palabras3[] = {'X', 'Y'};
291: int numPalabras3 = 2;
292:
293: int total3 = generarClaves("##0", palabras3, numPalabras3);
294:
295: printf("Total generado: %d claves (esperado: 40)\n", total3);
296: printf("Ejemplos:\n");
297:
298: /* Mostrar algunas claves de ejemplo */
299: for (int i = 0; i < 5 && i < total3; i++) {
300:     printf("  %s\n", resultados[i]);
301: }
302: printf("  ...\n");
303:
304: /* =====
305:  * EJEMPLO 4: Regla con caracteres literales
306:  * =====
307:  * Diccionario: ['a', 'b'] (2 palabras)
308:  * Regla: "pre#0fijo" â\206\222 "pre" + palabra + dÃ-gito + "fijo"
309:  *
310:  * Total esperado: 2 Ã\227 10 = 20 claves
311:  */
312: printf("\n--- EJEMPLO 4: Regla 'pre#0fijo' con diccionario ['a','b'] ---\n");
313:
314: char palabras4[] = {'a', 'b'};
315: int numPalabras4 = 2;
316:
317: int total4 = generarClaves("pre#0fijo", palabras4, numPalabras4);
318:
319: printf("Total generado: %d claves\n", total4);
320: printf("Ejemplos:\n");
321: for (int i = 0; i < 5 && i < total4; i++) {
322:     printf("  %s\n", resultados[i]);
323: }
324:
325: /* =====
326:  * RESUMEN FINAL
327:  * =====
328:  */
329: printf("\n===== \n");
330: printf("  RESULTADOS FINALES\n");
331: printf("===== \n");
332: printf("Ejemplo 1 (#0 con r,2): %d claves\n", total1);
333: printf("Ejemplo 2 (0#0 con A,B,C): %d claves\n", total2);
334: printf("Ejemplo 3 (##0 con X,Y): %d claves\n", total3);
335: printf("Ejemplo 4 (pre#0fijo con a,b): %d claves\n", total4);
336: printf("===== \n");
337: printf("â\234\223 Programa completado exitosamente\n");
338: printf("â\234\223 100% recursivo, sin break/continue\n");
339: printf("===== \n");
340:
341: return 0;
342: }
```